`@login_required` (see chapter 6 for more on this). Alternatively, if you want the venue information to be public, the view code needs to properly handle the possible lack of an associated profile. The following listing contains code for this latter approach.

Listing 7.22 A `venues()` view that handles unauthenticated visitors

```
# RiffMates/bands/views.py
...

def venues(request):
    all_venues = Venue.objects.all().order_by("name")
    profile = getattr(request.user, "userprofile", None)
    if profile:
        for venue in all_venues:
            # Mark the venue is "controlled" if the logged in user is
            # associated with the venue
            venue.controlled = profile.venues_controlled.filter(\
                id=venue.id).exists()
    else:
        # Anonymous user, can't be associated with the venue
        for venue in all_venues:
            venue.controlled = False
    ...
```

Don't assume that the .user attribute has a profile; use None if the user is anonymous.

If there is a profile, loop through the venues like before.

If there isn't a profile, the user can't control any venues. Set all attributes to False.

How paranoid should you be with files?

A lot happens behind the scenes for an image file to be uploaded and then presented to a user. A form gets used to submit a file, the file is stored on the server, and then the server serves the file in other pages. In the case of an image, the web server will set an appropriate mimetype for the file, giving the browser a hint about what is being served. Depending on the browser, that MIME type may be redundant, and anything pointed to by an `<img>` tag might get treated as an image.

There is a lot of *depending on* and *might* in that previous paragraph. A web server's features and configuration determine how it decides on a file's MIME type. By default, most web servers use the file extension as a hint. In theory, a user can upload a valid image file without the right file extension or with none at all. Django and your web browser try to help with this, but there are ways around it: never trust your users! In the case of an image file, the worst case is the image isn't interpreted as an image. Instead, you'll get the broken-picture symbol on the page. You could put extra code in the view handling the upload to ensure this doesn't happen, but this is a thin edge case with little consequences, so it likely isn't worth the extra work.

If you're allowing the user to upload files other than images, you might want to be more careful. A lot depends (there is that word again) on what you're doing with the uploaded file. If your service simply allows the file to be downloaded again, you may be able to get away with doing nothing with it. If you're doing data processing on the file, be very careful to check the file is what you expect. Good practice is to use third-party parsing libraries: someone else who is an expert in the file format is more likely to understand the corner cases than you are.